

Installing the NVIDIA Container Toolkit

Installation

Prerequisites

1. Review the [platform support documentation](#).
2. Install the NVIDIA GPU driver for your Linux distribution. NVIDIA recommends installing the driver by using the package manager for your distribution. For information about installing the driver with a package manager, refer to the [NVIDIA Driver Installation Quickstart Guide](#). Alternatively, you can install the driver by [downloading](#) a `.run` installer.

i Note

There is a [known issue](#) on systems where `systemd` cgroup drivers are used that cause containers to lose access to requested GPUs when `systemctl daemon reload` is run. Refer to the troubleshooting documentation for more information.

With `apt`: Ubuntu, Debian

i Note

These instructions [should work](#) for any Debian-derived distribution.

1. Install the prerequisites for the instructions below:

```
$ sudo apt-get update && sudo apt-get install -y --no-install-recommends \
  ca-certificates \
  curl \
  gnupg2
```

2. Configure the production repository:

```
$ curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor
&& curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-contai
sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-k
sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

Optionally, configure the repository to use experimental packages:

```
$ sudo sed -i -e '/experimental/ s/^#//g' /etc/apt/sources.list.d/nvidia-container-to
```

3. Update the packages list from the repository:

```
$ sudo apt-get update
```

4. Install the NVIDIA Container Toolkit packages:

```
$ export NVIDIA_CONTAINER_TOOLKIT_VERSION=1.20.0-1
sudo apt-get install -y \
    nvidia-container-toolkit=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
    nvidia-container-toolkit-base=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
    libnvidia-container-tools=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
    libnvidia-container1=${NVIDIA_CONTAINER_TOOLKIT_VERSION}
```

With **dnf**: RHEL/CentOS, Fedora, Amazon Linux

Note

These instructions should work for many RPM-based distributions.

1. Install the prerequisites for the instructions below:

```
$ sudo dnf install -y \
    curl
```

2. Configure the production repository:

```
$ curl -s -L https://nvidia.github.io/libnvidia-container/stable/rpm/nvidia-container
sudo tee /etc/yum.repos.d/nvidia-container-toolkit.repo
```

Optionally, configure the repository to use experimental packages:

```
$ sudo dnf config-manager --enable nvidia-container-toolkit-experimental
```

3. Install the NVIDIA Container Toolkit packages:

```
$ export NVIDIA_CONTAINER_TOOLKIT_VERSION=1.20.0-1
sudo dnf install -y \
    nvidia-container-toolkit-${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
    nvidia-container-toolkit-base-${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
    libnvidia-container-tools-${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
    libnvidia-container1-${NVIDIA_CONTAINER_TOOLKIT_VERSION}
```

With **zypper**: OpenSUSE, SLE

1. Configure the production repository:

```
$ sudo zypper ar https://nvidia.github.io/libnvidia-container/stable/rpm/nvidia-conta
```

Optionally, configure the repository to use experimental packages:

```
$ sudo zypper modifyrepo --enable nvidia-container-toolkit-experimental
```

2. Install the NVIDIA Container Toolkit packages:

```
$ export NVIDIA_CONTAINER_TOOLKIT_VERSION=1.20.0-1
sudo zypper --gpg-auto-import-keys install -y \
    nvidia-container-toolkit-${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
    nvidia-container-toolkit-base-${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
    libnvidia-container-tools-${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
    libnvidia-container1-${NVIDIA_CONTAINER_TOOLKIT_VERSION}
```

Configuration

Prerequisites

- You installed a supported container engine (Docker, Containerd, CRI-O, Podman).
- You installed the NVIDIA Container Toolkit.

Managing **config.toml**

The `nvidia-ctl config` command reads `/etc/nvidia-container-runtime/config.toml` by default. Without `--in-place` or `--output`, the command writes the updated configuration to standard output and does not change the source file.

Preview a change before you write it to the host:

```
$ sudo nvidia-ctk config \  
  --set nvidia-container-runtime.log-level=debug
```

Review the output. Then, add `--in-place` to update the source file:

```
$ sudo nvidia-ctk config --in-place \  
  --set nvidia-container-runtime.log-level=debug
```

You can specify `--set` more than once. The following command configures a log level and a log file in one update:

```
$ sudo nvidia-ctk config --in-place \  
  --set nvidia-container-runtime.log-level=debug \  
  --set nvidia-container-runtime.debug=/var/log/nvidia-container-runtime.log
```

For a list setting, separate elements with a colon. The following command makes `crun` the first low-level runtime candidate and retains `runc` as a fallback:

```
$ sudo nvidia-ctk config --in-place \  
  --set nvidia-container-runtime.runtimes=crun:runc
```

When the same key appears more than once, the last value takes effect. The command preserves existing settings when you change a different setting. Review the preview before every production update, especially when you use a custom file with `--config-file`.

Configuring Docker

1. Configure the container runtime by using the `nvidia-ctk` command:

```
$ sudo nvidia-ctk runtime configure --runtime=docker
```

The `nvidia-ctk` command modifies the `/etc/docker/daemon.json` file on the host. The file is updated so that Docker can use the NVIDIA Container Runtime.

2. Restart the Docker daemon:

```
$ sudo systemctl restart docker
```

Rootless Mode

To configure the container runtime for Docker running in Rootless mode, follow these steps:

1. Configure the container runtime by using the `nvidia-ctk` command:

```
$ nvidia-ctk runtime configure --runtime=docker --config=$HOME/.config/docker/daemon.
```

2. Restart the Rootless Docker daemon:

```
$ systemctl --user restart docker
```

3. Configure `/etc/nvidia-container-runtime/config.toml` by using the `sudo nvidia-ctk` command:

```
$ sudo nvidia-ctk config --set nvidia-container-cli.no-cgroups --in-place
```

Configuring containerd (for Kubernetes)

1. Configure the container runtime by using the `nvidia-ctk` command:

```
$ sudo nvidia-ctk runtime configure --runtime=containerd
```

By default, the `nvidia-ctk` command creates a `/etc/containerd/conf.d/99-nvidia.toml` drop-in config file and modifies (or creates) the `/etc/containerd/config.toml` file to ensure that the `imports` config option is updated accordingly. The drop-in file ensures that containerd can use the NVIDIA Container Runtime.

2. Restart containerd:

```
$ sudo systemctl restart containerd
```

Configuring containerd (for nerdctl)

No additional configuration is needed. You can run `nerdctl run --gpus=all`, with or without root. You do not need to run the `nvidia-ctk` command mentioned above for Kubernetes.

Refer to the [nerdctl documentation](#) for more information.

Configuring CRI-O

1. Configure the container runtime by using the `nvidia-ctk` command:

```
$ sudo nvidia-ctk runtime configure --runtime=crio
```

By default, the `nvidia-ctk` command creates a `/etc/crio/conf.d/99-nvidia.toml` drop-in config file. The drop-in file ensures that CRI-O can use the NVIDIA Container Runtime.

2. Restart the CRI-O daemon:

```
$ sudo systemctl restart crio
```

Configuring Podman



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2020-2026, NVIDIA Corporation.